PROJETO DE ENSINO	RELATÓRIO *[Descrição das atividades desenvolvidas]*	Formulário Nº 06

4.6 Atividades previstas no projeto

O projeto teve como objetivo construir e popular com atividades um sistema computacional em que estudantes pudessem realizar atividades de programação com correção automática. Para isto, deveriam ser construídos subsistemas para cadastro de questões por professores, para envio de respostas por estudantes e para correção automática das respostas e prover feedback rápido aos alunos. Além disto, deveria ser criada uma base de questões com diversidade de assuntos e contendo pelo menos uma centena de questões de programação. Para avaliar o sistema, ele deveria ser utilizado em pelo menos uma disciplina para coletar estatísticas de satisfação de docentes e discentes. Deveriam ainda ser medidos o número de critérios de qualidade atendidos pelo novo sistema, dentre aqueles listados no projeto.

Para alcançar estes objetivos, foram propostas as seguintes atividades:

- A1: Estudo e treinamento no uso das tecnologias que serão utilizadas no projeto.
- A2: Criação e gestão da infraestrutura de desenvolvimento com testes automatizados e integração e implantação contínuos.
- A3: Construção do subsistema de gestão de usuários (gestão de cadastro, login, perfis).
- A4: Construção do subsistema de gestão de cadastro de perguntas e respostas.
- A5: Construção do subsistema de correção automática.
- A6: Construção do subsistema de relatórios.
- A7: Criação da base de questões de programação.
- A8: Implantação e avaliação do sistema em uma disciplina.
- A9: Documentação do sistema e criação de manuais de utilização.
- A10: Coleta e análise de dados quanto a satisfação com o uso do sistema.

Todas as atividades com exceção da atividade A2 foram totalmente concluídas. A atividade A2 foi concluída parcialmente pois optou-se pela criação de um conjunto robusto e estável de funcionalidades no sistema em detrimento à criação de testes automatizados extensivos de todo o código-fonte. O código-fonte do sistema está disponível para download (a licença *open source* permite que a comunidade interna e externa à Ufes possam se beneficiar do projeto) em https://github.com/filipemtz/testr-2.0 e para utilização em http://200.137.66.71:8080/ (hospedado em um computador servidor do departamento de informática da Ufes). O sistema vem sendo utilizado com sucesso desde dezembro de 2024 para a realização de atividades avaliativas nas disciplinas de Estruturas de Dados, Programação Orientada a Objetos e Tópicos Especiais em Tecnologia e Inovação (ênfase em Redes Neurais Artificiais) dos cursos de Ciência e Engenharia da Computação. Como indicado na Figura 1, até o momento **181** estudantes estão cadastrados no sistema, foram criados **137** exercícios e foram submetidas mais de **6476** respostas a exercícios. Estes valores representam uma média de **36** exercícios de programação resolvidos por estudante. No semestre em que o sistema foi utilizado, a taxa de aprovação nas turmas de Estruturas de Dados, uma das disciplinas com alto nível de reprovação, foi de **75%**, sendo que os demais alunos foram reprovados por falta, isto é, **todos** os alunos que realizaram os exercícios e atividades avaliativas propostas foram aprovados.

181	137	6476	36
Estudantes	Exercícios Cadastrados	Respostas Enviadas	Exercícios Resolvidos em média por Estudante

Figura 1: Estatísticas de utilização do sistema Testr.

A Figura 2 traz a tela de login e cadastro de usuários que podem ser professores ou alunos. A Figura 3 traz a tela principal do sistema. Se o usuário for professor, ele pode cadastrar, editar ou remover cursos. Existem ainda funcionalidades para mudar a visibilidade dos cursos, isto é, torná-los visíveis ou invisíveis aos alunos. Por fim, é possível criar cópias de cursos para, por exemplo, utilizar uma base de questões em semestres consecutivos. Ao clicar em um curso, o usuário é levado à tela apresentada na Figura 4. A tela contém os tópicos do curso e, para cada um deles, uma lista de questões. Professores pode criar, editar, remover e controlar a visibilidade de tópicos e questões. Alunos podem apenas visualizar, mas não editar estas informações. A funcionalidade de controlar a visibilidade de seções permite que professores construam as listas de exercícios antes do início dos semestres letivos, mas apenas disponibilizem as listas (tornem elas visíveis) quando os tópicos forem estudados.

Ao selecionar a opção de criar uma questão, o professor é levado à tela apresentada na Figura 5. Cada questão é uma atividade de programação que contém título, descrição, linguagem de programação que deve ser utilizada para resolver a questão, data de entrega e restrições como quantidade máxima de memória que o programa poderá utilizar ou tempo máximo que ele poderá demorar para terminar. Ao criar uma questão, o professor pode adicionar arquivos contendo, por exemplo, informações adicionais sobre a questão, ilustrações ou códigos de exemplo. Para que as questões possam ser corrigidas automaticamente, o professor deve cadastrar ainda casos de teste que definem as saídas que o programa deve produzir a partir de um conjunto de entradas. Estes casos de teste especificam como o programa deve se comportar em diferentes situações.

A Figura 6 ilustra como as questões são apresentadas aos alunos. Nesta tela, os alunos podem enviar respostas que são avaliadas automaticamente pelo sistema de correção desenvolvido neste projeto. Para cada caso de teste, o sistema executa o programa e verifica se ele se comportou como esperado ou não. O resultado da avaliação é exibido na seção de relatório. Caso o programa falhe, são apresentadas informações explicando as causas da falha para que o aluno tenha fomento para corrigir a solução e fazer novas tentativas. Quando a questão é resolvida, o aluno recebe um balão verde, o que pode servir como incentivo e fator motivador.

A partir da tela de visualização do curso, professores podem acessar a página de configurações e relatórios do curso (Figura 7). Nesta página, outros podem ser adicionados como colabores do curso e alunos podem ser matriculados. No sistema desenvolvido, o professor é responsável por adicionar alunos nas turmas. Nesta tela, o professor pode ainda analisar o número de questões que cada aluno fez com sucesso, sem sucesso e o número de questões para as quais não foram feitas tentativas. Um relatório detalhado contendo o desempenho dos alunos por questão pode ser gerado em formato XLSX (arquivo do Microsoft Excel) para, por exemplo, atribuição de pontuação ao final de cursos.

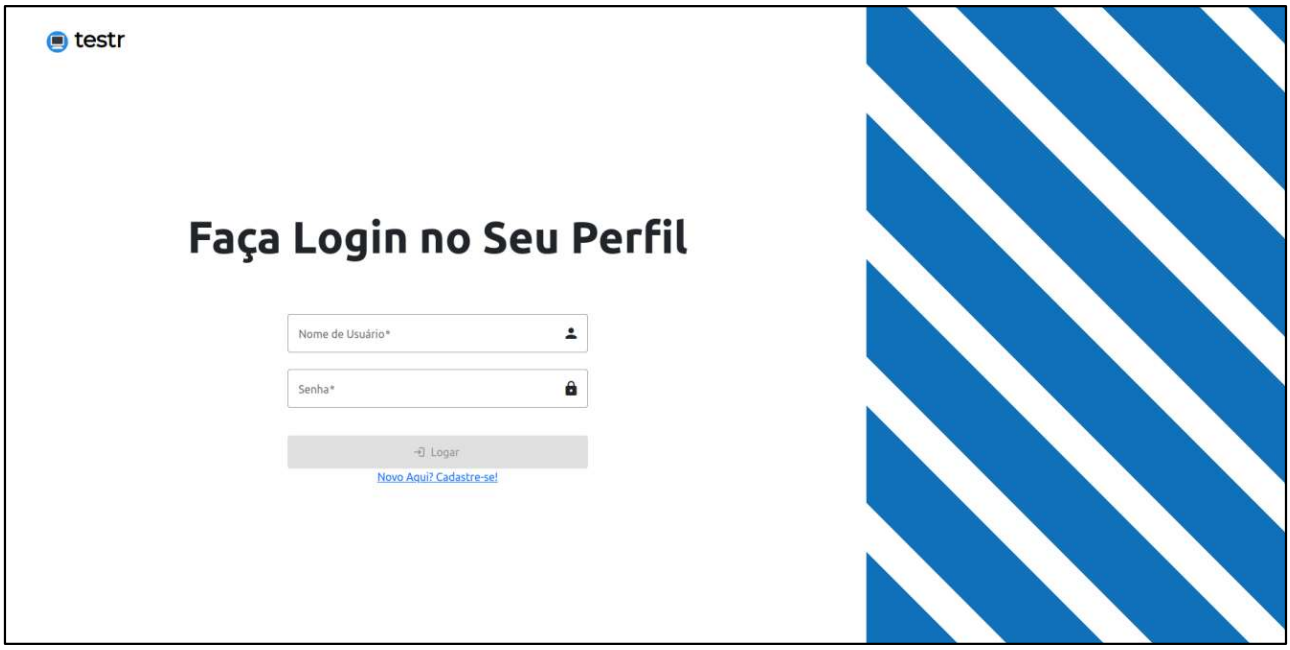

Figura 2: Tela de login e cadastro de usuários.

Figura 3: Tela principal do sistema. Professores podem gerenciar o cadastro de cursos, enquanto alunos podem apenas visualizar os cursos em que estão matriculados.

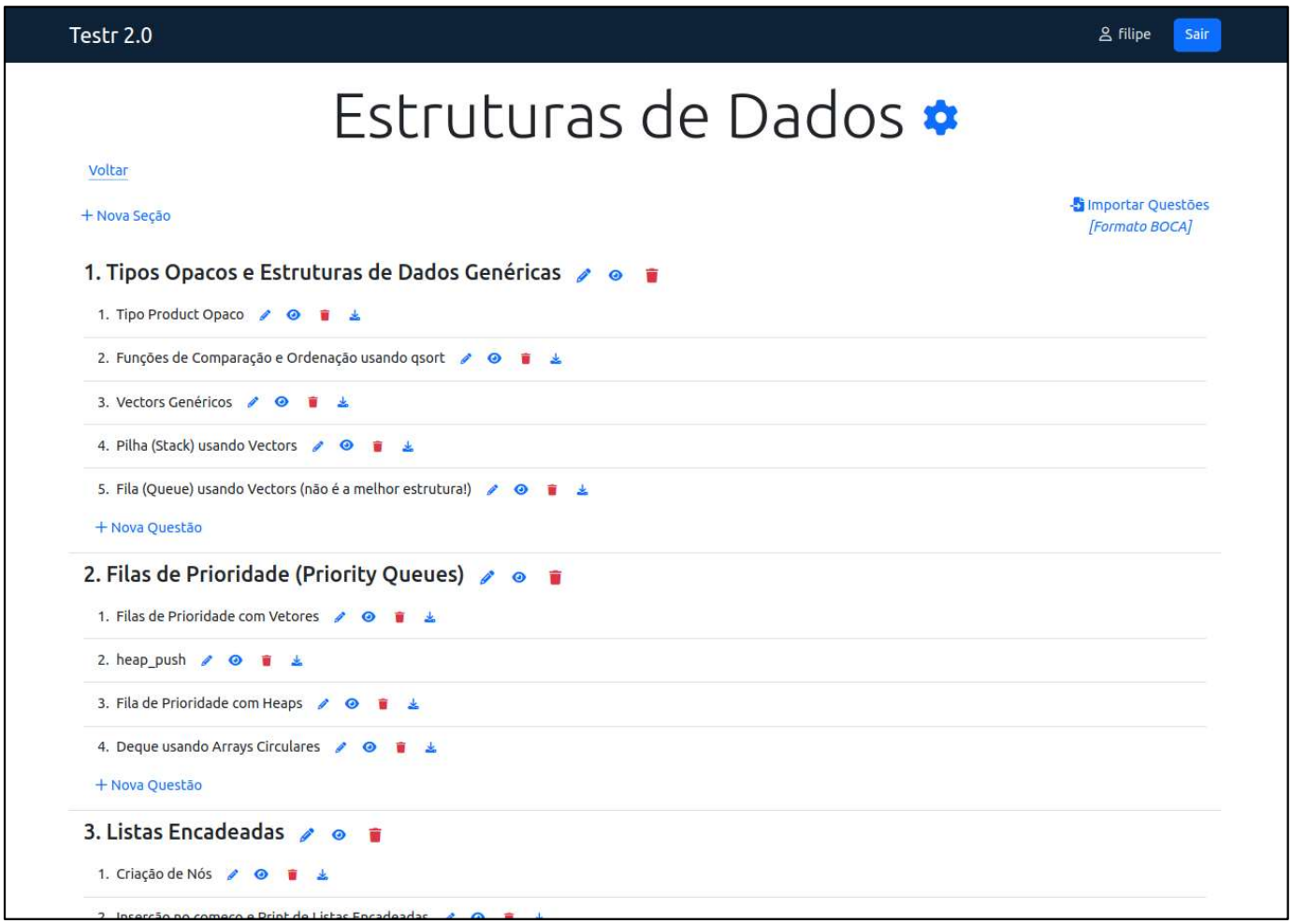

Figura 4: Tela de visualização de curso. Nesta tela, professores podem gerenciar o cadastro de tópicos de estudo e questões. Alunos podem acessar as questões para resolvê-las.

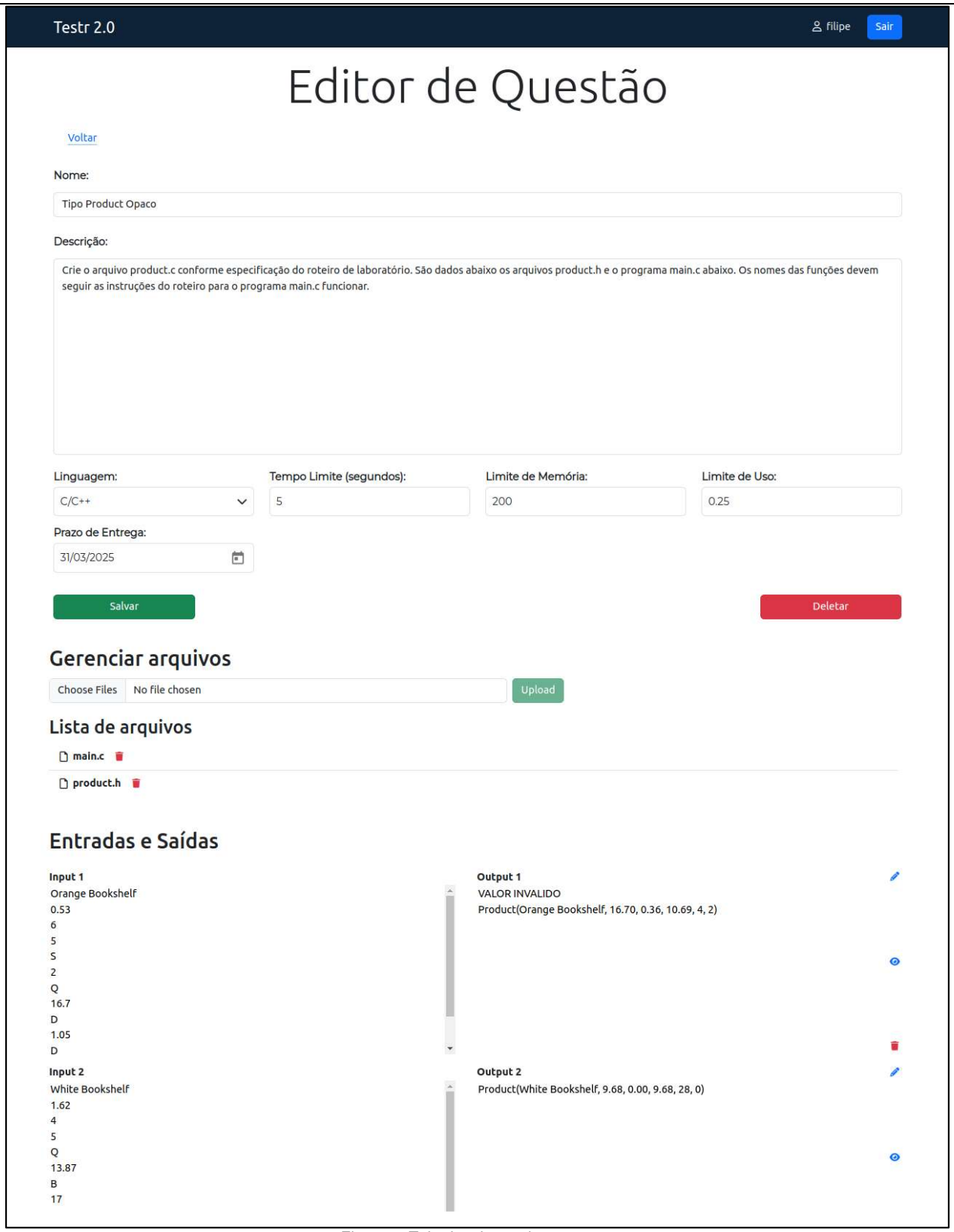

Figura 5: Tela de criação de questões.

Testr 2.0 filipe Sair

As Filas da Pizzaria

Voltar

Deadline: Dia 18 de dezembro de 2024 às 00:00 hora

Linguagem de Programação: C/C++

Descrição:
A sua tarefa é implementar parte do sistema de gestão de uma pizzaria. Pedidos chegam e são adicionados em uma fila. Quando o forno está livre, um pedido é retirado da fila, a pizza é assada e adicionada na fila de entregas. Quando um motoboy está disponível, uma pizza é retirada da fila de entregas e levada até o cliente. Para lidar com atrasos e obter informações sobre os pedidos, a equipe da pizzaria precisa ser capaz de visualizar os pedidos em diferentes posições das duas filas.

Crie um TAD Fila com operações de inserção, remoção e acesso usando índices com complexidade constante (sem uso de repetições). Para testar o TAD, escreva um programa que leia um número inteiro N e, em seguida, N comandos, um por linha. Os comandos podem ser dos seguintes tipos:

a. PEDIDO (e.g., "PEDIDO frango"): adiciona o pedido na fila do forno.
b. FORNO_LIVRE: retira o pedido mais antigo da fila do forno e adiciona na fila de entrega.
c. MOTOBOY: retira a pizza da fila de entrega e mostra na tela a mensagem ENTREGUE (e.g., "ENTREGUE frango").
d. INSPECAO_FORNO <índice: int>: mostra na tela o sabor da pizza no índice especificado da fila de pedidos aguardando o forno ficar livre.
e. INSPECAO_ENTREGA <índice: int>: mostra na tela o sabor da pizza no índice especificado da fila de entregas.

Observação: É obrigatório o uso de duas variáveis do tipo Fila para implementar a fila de pedidos e a fila de entregas.

Arquivos

Não existem arquivos

Entradas e Saídas

Entrada 1

```
3
PEDIDO frango
FORNO_LIVRE
MOTOBOY
```

Saída 1

```
ENTREGUE frango
```

Enviar Resposta

Choose File | No file chosen | Upload

Relatório

Arquivo	q1-fila.zip
Data da Submissão	Dia 15 de dezembro de 2024 às 23:52 horas
Status	Success
Relatório	

Reavaliar

Figura 6: Tela de solução de questões.

Figura 7: Tela da página de configuração e relatório dos cursos.

Por ser automático, o sistema de correção provê *feedback* imediato para os alunos permitindo que eles realizem atividades de forma autônoma, dentre e fora de sala de aula e em uma quantidade que seria inviável caso a correção dependesse do professor. A título de exemplo, na disciplina de Estruturas de Dados, cada aluno realiza mais de 50 atividades práticas de programação por semestre. Atividades podem ser desenhadas para serem incrementais, isto é, um projeto complexo pode ser subdividido em uma sequência de tarefas mais simples que podem ser resolvidas e avaliadas individualmente. Esta abordagem permite que estudantes recebam *feedback* quanto à corretude de cada etapa desenvolvida. A subdivisão em tarefas menores foi utilizada nos trabalhos práticos de programação e levou à um aumento na taxa de conclusão bem-sucedida dos trabalhos.

Sempre que necessário, o professor pode analisar as soluções enviadas pelos alunos para prover instrução individualizada aos estudantes, seja para ajudá-lo a encontrar o caminho para solução de uma tarefa mais desafiadora, ou para indicar como a solução pode ser modificada para cumprir mais critérios de qualidade (por exemplo, menor tempo de execução, menor uso de memória ou organização do código-fonte).

Os relatórios permitem avaliar o engajamento dos alunos com a disciplina ao longo do tempo. Tendo esta informação, incentivos podem ser dados aos alunos empenhados para que eles continuem se dedicando ao curso, e intervenções podem ser realizadas para identificar e solucionar fontes de desconexão junto a alunos que por algum motivo não estão realizando as atividades.

Foi realizado um questionário junto aos alunos da disciplina de Estruturas de Dados (2024/2) para identificar a sua perspectiva quanto à utilização do sistema. O questionário foi anônimo, não teve influência nas notas dos alunos e a ordem das respostas foi randomizada. Foram realizadas questões de múltipla escolha cujos resultados são dados nas Figuras 8 a 11. O formulário também apresentava caixas de texto nas quais os estudantes podiam se expressar de forma livre acerca de como o sistema se compara com outros em uso no departamento, sobre como foi a experiência de uso do sistema e possíveis sugestões de melhoria. A respostas são dadas nas Tabelas 1 e 2.

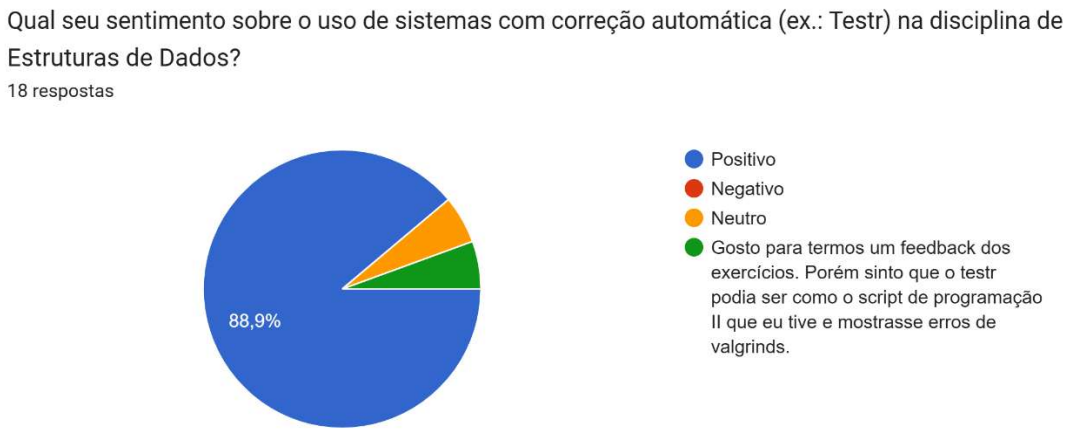

Figura 8: A maioria dos estudantes têm sentimento positivo quanto ao uso de sistemas com correção automática.

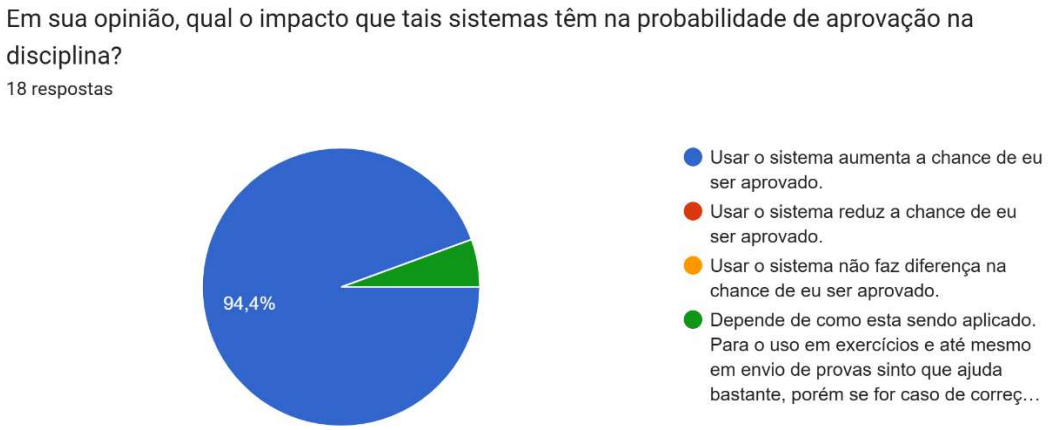

Figura 9: A maioria dos estudantes acredita que o uso de sistemas como o desenvolvimento neste projeto aumenta as chances de aprovação.

Figura 10: A maioria dos estudantes acreditam que aprendem mais usando o sistema.

Figura 11: A maioria dos estudantes considera que mesmo com o aumento considerável no número de tarefas proporcionado pelo sistema, o esforço adicional é equivalente ao necessário em outras disciplinas ou maior, mas gerenciável. Nenhum aluno considerou que o sistema gera aumento de esforço em um nível que gere esforço ou desgaste.

Tabela 1: Respostas à pergunta "**Qual *sistema você prefere, BOCA ou Testr? Por quê?***" via caixa de texto.
(O sistema BOCA é um sistema similar em utilização no departamento de informática)

- Testr
- Acho que a mecânica de ranking do BOCA motiva bastante, mas de resto o Testr é bem melhor, pelo design amigável, descrição do problema com inputs e outputs, sem precisar baixar PDFs, registros de data de envio, com a possibilidade de monitoramento pessoal de progresso na disciplina, além de velocidade de correção muito superior com o Testr 2.0
- testr, porque fornece mais informações sobre seu erro.
- Testr. BOCA NAO TEM LOG, TA ERRADO! PORQUE ? ELE NAO EXPLICA! "CONSERTA AI MEU PARSA" MUDA CODIGO, FAZ ZIP ENVIA E TA ERRADO DENOVO, É O MESMO ERRO OU DIFERENTE?
- Testr. Mostra os enunciados, inputs e outputs sem a necessidade de baixar arquivo. Mais prático.
- Testr. É um sistema com a interface e utilização mais intuitivos para com o usuário.
- Nunca usei o BOCA.
- Testr, oferece mais mecanismos, como por exemplo, quando o codigo falha mostrar a saida gerada.
- Prefito muito mais a interface do Testr, porém gostaria que ele gerasse um LOG e uma saída com o resumo do valgrind.
- Testr, além da interface ser mais limpa é muito bom que ele gera um resultado mais detalhado e me ajuda a encontrar o meu erro, no BOCA é apenas um "certo" ou "errado" e não exibe nenhum relatório o que é bem chato. A única coisa que gostaria é que o Testr o balãozinho nao ficasse só quando clicamos na questão enviada e pudesse ser exibido em miniatura do lado do nome já na listagem de atividades pois quando esqueço o nome da atividade que parei tenho que entrar um a um pra ver qual já enviei e essa visualização deixaria mais prático.
- Testr. A correção é melhor.
- Só conheci o Testr
- Para mim é diferente, porém o Testr tem uma interface mais amigável.
- Nunca utilizei o BOCA
- Testr. Acho o BOCA simples até demais.

Tabela 2: Respostas à pergunta "***Use este campo para fazer comentários gerais, sugestões ou críticas sobre o uso de sistemas de correção automática em disciplinas de programação e, em especial, sobre o sistema Testr***" via caixa de texto.

- Acho fundamental o uso de alguma, pois, sem elas, talvez eu não me sentiria motivado de fazer exercícios fazendo correções manuais e acreditaria, erroneamente, que já teria domínio da disciplina. Sobre o Testr em particular, ele motiva ainda mais por ter um design mais agradável e convidativo.
- Eu sugeriria abrir um campo de ranking, mesmo se fosse anônimo (ou pelo menos anônimo da metade pra baixo)
- O testr 2.0 não possibilita ver os resultados de todas as respostas enviadas, apenas da ultima. Apesar disso acho muito bom o sistema para o aprendizado da matéria.
- testr perfeito. o ruim é ter que atualizar toda hora e arrastar pra baixo
- Podia ter um botão no início da página do enunciado que redireciona pro quadro de envios de código. Às vezes, eu estava com bastante pressa pra saber o que errei e tinha que scrollar centenas de inputs até a tabela.
- Foi uma surpresa agradavel, o sistema e facil de se operar, raramente tenho problemas ou dificuldades, diferentemente do boca.
- Sinto que é necessário sistemas de correção como o Boca e o Testr. Mas os dois ainda tem de melhorar em certos quesitos. Sinto que se existisse uma mescla entre os dois. Interface e gerenciamento de atividades do Testr com a saída/correção do Boca teríamos que eu não vejo como fazer muitas criticas.
- Gosto mais do design da 2.0 mas tive problema ao logar na única vez que usei (submissão da prova) de resto não usei mais então não consigo opinar tanto. O Testr é muito melhor que o boca, gosto muito
- Uma ferramenta de apoio e organização útil, que ajuda o aluno manter em mente o que precisa ser feito em determinado prazo. Gostaria que o testr disponibilizasse arquivos de entrada e saída.
- Apenas um detalhe: no Testr poderia ter um ícone de "check" ao lado do nome da atividade na tela de listagem de atividades, indicando que aquela atividade já foi concluída (existe pelo menos um envio com status "Success").

A questão apresentada na Figura 8 teve como objetivo avaliar o sentimento geral do estudante acerca do uso de sistemas com correção automática e 88.9% têm sentimento positivo. Note que a pergunta não focou no sistema desenvolvido e engloba outros sistemas utilizados nos cursos de Ciência e Engenharia da Computação. Segunda a Figura 9, a grande maioria dos estudantes, 94.4%, acredita que utilizar estes tipos de sistema aumenta a chance de aprovação nas disciplinas. Este sentimento é compartilhado por professores que ministram disciplinas de programação dos semestres iniciais. Infelizmente, não é viável avaliar quantitativamente a variação na taxa de aprovação dado que não existe consistência no nível de dificuldade das avaliações e no perfil de conhecimento *à priori* dos estudantes. O aumento no número de institutos federais gerou aumento no número de estudantes que estudaram programação no ensino médio e desenvolveram conhecimentos sólidos de matemática e física. Em adição a este fator, a variação no perfil dos estudantes ao longo dos anos e a presença de alunos repetentes nas turmas torna difícil a comparação das taxas de aprovação entre semestres e, principalmente, com décadas anteriores. Por fim, vale notar que a taxa de aprovação e a quantidade de conhecimento adquirido não são valores diretamente conectados. Estudantes podem aprender mais e a taxa de reprovação pode se manter constante porque, por exemplo, professores podem aumentar o nível de dificuldade ou aprofundar assuntos em resposta ao maior conhecimento *a priori* dos estudantes.

A Figura 10 mostra que 77.8% dos estudantes acreditam que utilizar sistemas com correção automática faz com que eles aprendam mais em comparação com a não utilização de tais sistemas e 22% acreditam que usar estes sistemas não faz diferença no quanto eles aprendem. A principal vantagem no uso destes sistemas é o aumento substancial no número de atividades práticas realizadas. Uma preocupação, contudo, é que o aumento da carga de atividades poderia atrapalhar outras disciplinas ou causar níveis preocupantes de desgaste e *stress* considerando que o estudante está

inserido em um grande número de disciplinas. Estas preocupações se tornam mais relevantes em um contexto de aumento dos casos de síndromes como ansiedade, depressão e *burnout* entre jovens. A Figura 11 aborda esta questão e 61.1% dos estudantes afirmam que o esforço necessário com o uso do sistema é equivalente àquele necessário em outras disciplinas e 27.8% dizem que o esforço é grande, mas gerenciável, isto é, ele não atrapalha outras disciplinas ou gera desgaste.

4.7 Atividades não realizadas

A atividade A2 (criação e gestão da infraestrutura de desenvolvimento com testes automatizados e integração e implantação contínuos) não foi realizada para priorizar-se a criação de funcionalidades robustas.

4.8 Atividades não previstas, porém realizadas

Foram criadas mais questões que o previsto e o sistema foi utilizado em várias disciplinas.

PROJETO DE ENSINO	RELATÓRIO *[Avaliação dos resultados]*	Formulário Nº 06.1

4.8 Avaliação dos resultados

O projeto foi concluído com sucesso considerando que os objetivos definidos foram alcançados e o sistema vem sendo utilizado em múltiplas disciplinas. Os alunos que participaram do projeto utilizando o sistema foram avaliados como indicado a seguir. Dentre as atividades avaliativas realizadas no semestre, 10% da nota era relativa à realização dos exercícios de programação utilizando o sistema desenvolvido neste projeto. O sistema também foi utilizado em trabalhos e provas para os estudantes testarem seus programas, mas, nestes casos, a avaliação foi feita manualmente pelo professor para garantir uma análise mais detalhada das respostas. Nas turmas de Estruturas de Dados (2024/2), os alunos realizaram em média **67%** dos exercícios. Desconsiderando alunos reprovados por falta, a taxa aumenta para **85%** de exercícios resolvidos em média. A lista de estudantes cadastrados no sistema (participantes do projeto) é enviada em uma planilha em anexo.

Os monitores foram avaliados considerando a qualidade técnica das entregas, cumprimento de prazos e proatividade. Ambos os monitores alcançaram desempenho satisfatório, como pode ser atestado pela qualidade do sistema desenvolvido. Como indicado na Seção 4.6, o projeto foi avaliado usando um formulário do *Google Forms* enviado para os estudantes que utilizaram o sistema nas turmas de Estruturas de Dados (2024/2). Os resultados são descritos na referida seção.

4.9 Contribuição do projeto para o ensino de graduação

O projeto contribuiu para o processo de ensino e aprendizagem em disciplinas de programação por permitir que estudantes realizem uma grande quantidade de exercícios e recebam *feedback* imediato, algo que não seria possível sem a automatização de parte do processo. O alto índice de aprovação na disciplina de Estruturas de Dados no semestre 2024/2 (100% desconsiderando reprovados por falta), pode em parte ser atribuído ao grande número de exercícios aos quais os alunos foram expostos. Vale enfatizar que a disciplina é listada como tendo alto nível de reprovações.

Data: 30/10/2025

_____
Coordenador